

# IoT - Round 2

**Instructions:** Read the situation carefully. Build the system described in 'What to Build' using only the components in your kit. Use Arduino Uno as the microcontroller. No internet or cloud connectivity required.

## Scenario 1

### The Midnight Intruder Alarm

Team 1

**Component Kit:** LDR · PIR Motion Sensor · LED Red · Piezo Capsule · Pushbutton

#### The Situation

*The college lab is locked after 6 PM — but someone has been sneaking in undetected. The warden installs a lock, but it gets bypassed. CCTV is expensive. The warden reaches out to the IoT lab team and says: "I need something that detects motion in the dark and raises an alarm — but lets me silence it quickly if it's just me checking in."*

#### What to Build

1. Read LDR via analogRead to determine dark/light. When dark, monitor PIR for motion.
2. Motion detected → red LED flashes rapidly + piezo alarms.
3. Pressing the pushbutton within 5 seconds silences the alarm.
4. If button is NOT pressed within 5 s, alarm continues for a full 20 seconds.
5. After alarm ends, system re-arms automatically. Use millis() — no delay().

## Scenario 2

### The Touchless Hospital Dustbin

Team 2

**Component Kit:** Ultrasonic Sensor · Small Servo Motor · LED Green · Piezo Capsule · Resistors

#### The Situation

*In the college medical room, bins are constantly being touched by multiple people — spreading germs during flu season. A nursing student raises a concern: "We need bins that open themselves when you bring your hand close. No touch, no infection." The challenge is to make the lid movement smooth and controlled — not a sudden snap.*

#### What to Build

6. Poll the HC-SR04 every 150 ms. When a hand is detected within 15 cm, smoothly sweep the servo from 0° (closed) to 90° (open) — one degree at a time, 8 ms between steps.
7. Hold open for 4 seconds, then sweep back to 0° at the same speed.
8. Green LED stays on while lid is open.
9. One short beep when opening, two short beeps when closing.
10. Block re-trigger while lid is already moving or open.

|                |                                                   |        |
|----------------|---------------------------------------------------|--------|
| Scenario<br>3  | The Seminar Hall Occupancy Counter                | Team 3 |
| Component Kit: | IR Sensor (×2) · LED Green · LED Yellow · LED Red |        |

### The Situation

*The college seminar hall seats 80 people but is never managed properly. Some events are dangerously overcrowded while others are half-empty. The Events Committee asks: "Can we have a live headcount at the door — something that tells us exactly how many people are inside right now, without cameras or manual counting?"*

### What to Build

11. Mount IR sensor A and B 5 cm apart at the door frame.
12. Track which sensor triggers first: A then B = entry (count++), B then A = exit (count--).
13. Allow a 300 ms window to detect the second sensor after the first fires.
14. Display occupancy on 3 LEDs: green = low (0–3), yellow = medium (4–7), red = high (8+).
15. A short beep on each valid entry or exit. Count must not go below 0.
16. Log direction and count to Serial on every change.

|                |                                                          |        |
|----------------|----------------------------------------------------------|--------|
| Scenario<br>4  | The Smart Staircase Lighting                             | Team 4 |
| Component Kit: | PIR Motion Sensor (×2) · LED Green · LED Red · Resistors |        |

### The Situation

*The hostel staircase is completely dark after 10 PM. A student broke their wrist tripping in the dark last semester — the switch is at the wrong end and nobody bothers to flick it. The hostel warden wants lights that respond automatically, stay on long enough for safe passage, and need no switches at all.*

### What to Build

17. Two PIR sensors simulate top and bottom of a staircase.
18. PIR-top triggers → green LED on for 20 s; resets timer if motion is detected again within the window.
19. PIR-bottom triggers → red LED on for 20 s with the same re-trigger logic.
20. Both LEDs can be on simultaneously — they operate fully independently.
21. No blocking code — use millis() for both independent 20 s timers.
22. Serial-print which PIR triggered and each LED's remaining time every 5 s.

| Scenario<br>5  | The Smart Pedestrian Crossing                                            | Team 5 |
|----------------|--------------------------------------------------------------------------|--------|
| Component Kit: | Pushbutton · Ultrasonic Sensor · Small Servo Motor · LED Green · LED Red |        |

### The Situation

*A narrow lane through the college campus has become dangerous. Vehicles speed through while pedestrians step out from between parked cars. Two near-misses in the same month forced the administration to act. A proper traffic light system would take months of approval. You have a weekend and five components.*

### What to Build

23. Idle state: red LED on (stop), servo at 0° (barrier down).
24. Pedestrian presses button → check HC-SR04 for vehicle (distance < 40 cm = vehicle present).
25. Vehicle present → red LED flashes fast, deny crossing, recheck every 500 ms for up to 10 s.
26. No vehicle → green LED on + servo sweeps to 90° (barrier up). Hold crossing open for 7 s.
27. Servo sweeps back to 0°, green off, red on.
28. Serial-log each button press, vehicle detection result, and crossing duration.

| Scenario<br>6  | The Malfunctioning Traffic Signal                                           | Team 6 |
|----------------|-----------------------------------------------------------------------------|--------|
| Component Kit: | LED Yellow · LED (bright white) · Pushbutton · Piezo Capsule · Relay Module |        |

### The Situation

*The traffic signal at the college entrance has broken down. It's stuck on red and hasn't changed in two days. Vehicles are piling up and pedestrians are jaywalking dangerously. The college principal calls the IoT lab team: "I need a temporary working traffic signal installed by tomorrow morning — one that runs automatically but lets a traffic officer override it manually with a button."*

## What to Build

29. Normal operation: cycle automatically — white LED on (go) for 8 s → yellow LED blink 3 times (slow down) → relay clicks off (stop/red, simulated) → repeat.
30. Piezo beeps once at every phase change so people nearby can hear the signal switch.
31. Pushbutton = officer override: immediately jumps to stop phase (relay OFF, yellow LED off, white LED off) and holds for 15 s before resuming auto-cycle.
32. During override: yellow LED flashes rapidly to indicate manual control is active.
33. Use millis() for all phase timing — no delay(). Serial-print current phase and time remaining every second.

---

|                |                                                                   |        |
|----------------|-------------------------------------------------------------------|--------|
| Scenario<br>7  | The Forgotten Classroom Lights                                    | Team 7 |
| Component Kit: | LDR · Relay Module · LED Yellow · LED (bright white) · Pushbutton |        |

## The Situation

*Every evening the college classrooms are left with lights blazing — the last student out never bothers to switch them off. The electricity bill has doubled. The facilities manager is furious: "I want the lights to turn off automatically when it gets dark outside and nobody is in the room — but I still want a manual override button for when someone genuinely needs to keep them on."*

## What to Build

34. Read LDR every 1 s using millis(). When ambient light drops below threshold ( $< 300$  ADC) → relay ON, simulating room lights turning off (or fan stopping).
35. When light is above threshold (bright, daytime) → relay stays OFF (lights not needed).
36. White LED mirrors relay state — on when relay is on.
37. Yellow LED = system status indicator, always on to show the controller is active.
38. Pushbutton toggles a manual override flag: when override is ON, relay is forced OFF regardless of LDR (lights stay on). Press again to release override.
39. Serial-print LDR value, relay state, and override flag every 2 s.

---

|                |                                                                              |        |
|----------------|------------------------------------------------------------------------------|--------|
| Scenario<br>8  | The Runaway Workshop Fan                                                     | Team 8 |
| Component Kit: | Potentiometer 10k $\Omega$ · DC Motor 5V · Transistor · LED Yellow · LED Red |        |

## The Situation

*The college workshop bench fan has no speed limit. Students routinely crank it to maximum, and last semester the plastic blades cracked under stress, sending a fragment across the room. Nobody was hurt — barely. The HOD has mandated: the fan must not exceed a safe speed, and must slow itself down automatically if a student tries to go too fast.*

### What to Build

40. Read potentiometer every 200 ms. Map ADC (0–1023) to PWM (0–255) and drive DC motor via NPN transistor.
41. Yellow LED = slow zone (PWM < 100). Both LEDs off = medium (100–200). Red LED = fast (> 200).
42. When PWM > 220 (danger zone): automatically cap PWM at 180 — override the potentiometer input.
43. Red LED rapid flash + piezo 3000 Hz warning beep every 2 s during override.
44. When pot is dialled back below 200, restore normal pot control and clear override.
45. Serial-log pot value, PWM applied, speed zone, and override flag every 500 ms.